

skullbase-corridor: Finite-instrument corridor geometry analysis for skull-base research in 3D Slicer

Article metadata

- **Article type:** SoftwareX software article
- **Authors:** Abhinav Bachu*, Raghav Rajesh, and Anand V. Germanwala
- **Affiliation:** Department of Neurological Surgery, Loyola University Chicago Stritch School of Medicine, Maywood, Illinois, USA
- **Corresponding author:** Abhinav Bachu (*)
- **Corresponding-author email:** abachu@luc.edu
- **Software release cited by this article:** version 0.2.1
- **Repository URL:** <https://github.com/abachu2005/skullbase-corridor>
- **Immutable release tag/commit:** `v0.2.1 / a0c6933258f60e370747cfd4096b9445eff8a572`
- **Archived software identifier/DOI:** [10.5281/zenodo.23244307](https://doi.org/10.5281/zenodo.23244307)

Abstract

skullbase-corridor is open research software for reproducible, surgeon-supervised exploration of rigid skull-base approach corridors in physical image coordinates. It represents target samples, protected structures, finite circular portals, and finite rigid instruments, and records evaluated trajectories, insertion depths, clearances, coverage set operations, and explicit rejection or abstention reasons. The package combines a typed Python geometry engine, command-line and desktop interfaces, and a 3D Slicer module for image, segmentation, markup, linked-slice, three-dimensional, and report workflows. A deterministic worked example demonstrates feasible trajectories and distinct sampled coverage for two configured approaches. Verification includes analytical fixtures, independent geometric references, cross-platform automated tests, synthetic desktop workflows, and bounded public-data execution. The software supports research and method development; it is not clinically validated.

Keywords

research software; computational geometry; medical imaging; 3D Slicer; finite instruments; skull base; trajectory planning

Code metadata (mandatory)

Field	Value
C1 Current code version	v0.2.1
C2 Permanent link to code/ repository used for this code version	https://doi.org/10.5281/zenodo.23244307 ; source repository: GitHub
C3 Permanent link to Reproducible Capsule	N/A; no executable reproducible capsule has been deposited
C4 Legal Code License	Apache License 2.0 (Apache-2.0)
C5 Code versioning system used	Git

Field	Value
C6 Software code languages, tools, and services used	Python ≥ 3.11 ; 3D Slicer scripted module; Hatchling; GitHub; GitHub Actions
C7 Compilation requirements, operating environments, and dependencies	Python ≥ 3.11 . Release CI passed on Ubuntu, macOS, and Windows with Python 3.11 and 3.12. Core dependencies: NumPy, SciPy, nibabel, SimpleITK, pydicom, Pydantic, and Typer. Optional desktop dependencies: PySide6, pyqtgraph, and VTK. The scripted module requires 3D Slicer 5.12.
C8 Link to developer documentation/manual	README.md, docs/USER_GUIDE.md, and slicer/README.md
C9 Support email for questions	abachu@luc.edu

Issues may additionally be reported at github.com/abachu2005/skullbase-corridor/issues.

1. Motivation and significance

Computational descriptions of a surgical corridor can obscure distinctions that materially change a geometric result. An infinitely thin ray is not a finite-width instrument; an instrument has finite length; an oblique instrument must fit through an aperture; an alternative route is not necessarily available simultaneously with another route; and an unrepresented structure is not demonstrated free space. A sampled failure to reach a target is also not proof of physical or operative inaccessibility.

`skullbase-corridor` was developed to make these assumptions explicit and inspectable. The intended users are technically supported surgeons and researchers studying configured geometric scenarios. Portal locations, target samples, protected structures, registration, segmentations, instrument dimensions, and review state remain inputs requiring human review. The software does not convert these inputs into clinical ground truth.

The contribution is a reusable software representation of finite-instrument corridor geometry with traceable witnesses and conservative handling of incomplete information. Existing medical-image visualization and annotation functions are used through 3D Slicer [1]. The package adds typed corridor semantics, finite insertion and aperture constraints, exact-path state reporting, approach-specific set comparisons, incomplete-anatomy abstention, and checksummed exports. It supports reproducible software experiments without asserting that the configured openings are anatomically appropriate or that a geometrically feasible path is surgically safe.

2. Related work

3D Slicer provides an extensible environment for segmentation, registration, visualization, and image-guided-intervention research [1, 2]. Procedure-specific modules have supported brain laser-ablation planning, pedicle-screw trajectories, implant navigation, and stereoelectroencephalography workflows [3, 4, 5, 6]. These systems establish the value of integrating planning with image review, but do not address finite-instrument skull-base aperture geometry.

Computer-assisted neurosurgical trajectory planning has been studied for deep brain stimulation, keyhole procedures, and intracranial electrodes [7, 8, 9, 10, 11, 12]. The closest methods rank straight centerlines or cylindrical safety envelopes according to distances from segmented hazards. Shamir et al. map candidate cranial entries and placement uncertainty [9], while Bériault et al. use a cylindrical trajectory model [8]. The present software therefore does not claim novelty merely for assigning a nonzero diameter to a trajectory. Its distinction is the joint representation of finite insertion length, shaft and tip radii, a constrained portal aperture, target reach, exact retained witnesses, and alternative versus simultaneous coverage.

Quantitative skull-base anatomy commonly compares area of exposure, corridor depth, angle of attack, surgical freedom, or working volume in cadaveric specimens [13, 14, 15, 16, 17]. Those measurements characterize named approaches and operative exposure. They do not directly compute a patient-specific feasible set of finite rigid-instrument poses against represented anatomy. `skullbase-corridor` complements rather than replaces that literature: it evaluates a declared geometric model and does not infer operative exposure or maneuverability.

TotalSegmentator and its nnU-Net foundation can reduce the effort required to construct image-derived models [18, 19], but general-purpose labels do not supply all fine skull-base neurovascular boundaries. The software therefore preserves missing structures as unavailable instead of interpreting automated segmentation as complete anatomy.

3. Software architecture and functionality

3.1 Components and data flow

The `src/skullbase_corridor` Python package separates domain models, image input/output, geometry primitives and engines, analysis, application services, desktop views, anatomy adapters, and exports. Typed Pydantic contracts define cases and results. The command-line interface supports synthetic-case generation, analysis, and environment reporting. The optional Qt/VTK desktop application provides linked anatomical slices, three-dimensional inspection, editable configuration, asynchronous analysis, and export.

The scripted Slicer module integrates CT selection, markups, segmentation review, candidate display, exact evaluated paths, linked two- and three-dimensional views, scene persistence, and report export. A separate bounded bridge can export selected Slicer segments on the full CT grid, convert Slicer’s array order to engine order, retain the world-RAS affine, and import a saved coverage segmentation. Targets above the bridge’s 2,000-foreground-voxel limit are rejected rather than silently subsampled.

The packaged Slicer workflow launches a managed TotalSegmentator [18] process, using CPU or a compatible configured accelerator. Model predictions are labeled as predictions and require review. The geometry engine consumes reviewed image-derived structures independently of how those structures were produced.

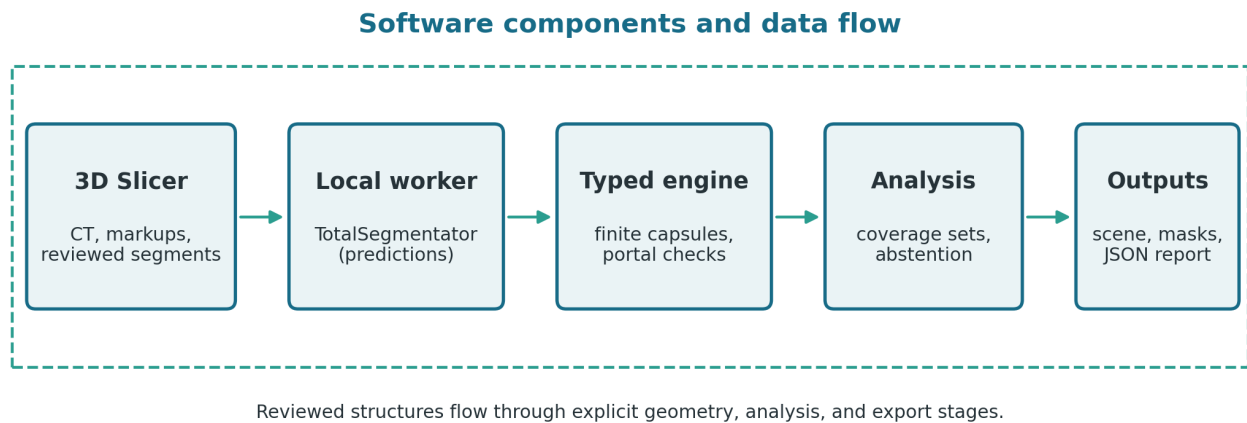


Figure 1. Software architecture and data flow from image review through model-assisted segmentation, finite-instrument analysis, coverage comparison, and export.

3.2 Coordinates and input representations

All engine geometry is represented in millimetres in a declared physical frame. Affine matrices map voxel indices to physical positions. Image loaders normalize declared RAS/LPS conventions and spatial units to RAS millimetres, and incompatible frames are rejected rather than silently aligned. Registration and resampling are explicit review operations.

Targets can be supplied as physical-coordinate points or mask-derived voxel centres. Complete voxel representations may carry physical cell weights computed from the affine determinant. Sparse samples support count-based coverage only and are not interpreted as tumour-volume percentages. Protected geometry can be represented by analytical spheres or affine voxel masks. Unsupported geometry, including the current unsupported mesh input route, causes abstention rather than being treated as empty space.

3.3 Finite-instrument reach and aperture model

For entry point $\mathbf{e}$, unit direction $\mathbf{d}$, and target sample $\mathbf{x}$, required axial insertion is

$$t = (\mathbf{x} - \mathbf{e}) \cdot \mathbf{d}$$

and perpendicular distance from the trajectory is

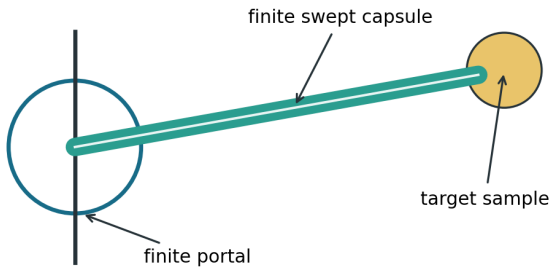
$$\|(\mathbf{x} - \mathbf{e}) - t\mathbf{d}\|.$$

A target sample is a reach candidate only when insertion is nonnegative, does not exceed instrument length, and the perpendicular distance is no greater than the working-tip radius plus configured target tolerance. Shaft radius is not added to target tolerance.

The inserted instrument is modeled as a finite capsule from the entry point to the insertion depth required for the target. Collision and aperture checks conservatively use the larger of shaft radius and working-tip radius. A portal is a finite circular disk with a declared forward normal. The software checks forward traversal and conservatively tests whether the oblique shaft footprint fits the aperture. This is not a manufacturer-specific, articulated, deformable, or tissue-interacting instrument model.

Synthetic schematic (not patient anatomy; not to scale)

A Finite-instrument model



B Exact-path state reporting

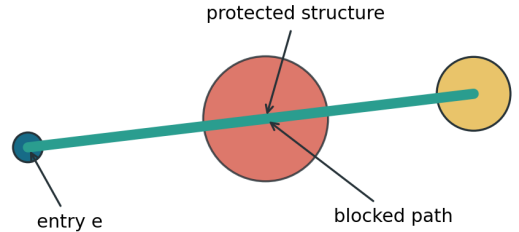


Figure 2. Synthetic finite-instrument geometry schematic. (A) A finite swept capsule traverses a finite portal toward a target sample. (B) Exact-path evaluation retains collision states against represented protected geometry. This schematic is not patient anatomy and is not to scale.

Analytical sphere clearance is computed directly. Voxel masks represent closed affine cells. The coarse voxel backend uses conservative physical-space bounds and interpolation guards; optional near-boundary refinement checks candidate cells under a finite budget. Exhausting that budget preserves conservative blockage. Geometry required outside a protected mask field of view triggers the configured out-of-field policy.

3.4 Abstention, sampling, and comparison semantics

Missing, unknown, unsupported, or out-of-field protected anatomy can produce abstention. An explicit exploratory policy may permit a conditional result, but affected quantitative claims remain incomplete or suppressed. These policies do not detect every missing structure, segmentation error, incorrect portal, or unmodeled tissue.

The engine evaluates a configured angular grid and may add target-directed and budgeted adaptive directions. This search is finite and not exhaustive. Each evaluated trajectory retains its direction, target indices, insertion depths, clearance information, and rejection or abstention reason. Consequently, “not reached” means that no evaluated path reached the sample; it does not mean that no path exists.

Reached target-index sets are retained separately by approach. Union, intersection, and incremental coverage are computed as set operations to avoid double counting. Simultaneous-access analysis additionally enforces angular separation and finite capsule-to-capsule clearance. Alternative-path union coverage is not interpreted as simultaneous feasibility.

Supported unobstructed centre-entry cases use geometry-aware interval-union integration for feasible solid angle. Protected-structure cases use spherical-cap grid quadrature with Voronoi area weights; added target-directed and adaptive witnesses receive no quadrature weight. Unsupported or nonconverged measurements are omitted. The software makes no general one-percent angular-accuracy claim.

4. Illustrative examples

4.1 Data-free synthetic workflow

A reader can generate and analyze a deterministic synthetic configuration:

```
python -m pip install -e '.[desktop,test]'  
skullbase-corridor synthetic case.json  
skullbase-corridor analyze case.json result.json  
skullbase-corridor doctor
```

The output records normalized configuration, software and schema versions, source and result checksums, evaluated trajectories, coverage sets, and abstention reasons. The bundled desktop demonstration can be opened without patient data or network access. Changing an analysis parameter invalidates the displayed result until analysis is rerun.

4.2 Reviewed segmented workflow in 3D Slicer

In the reviewed-segment workflow, a user loads a CT and aligned segmentations, identifies target and entry geometry, selects target and protected-anatomy segments, optionally supplies approach-specific bone-removal masks, configures portal and instrument dimensions, and runs the external geometry engine. The output segmentation categorizes target samples as approach-only, shared, sampled-unreached, or unavailable. These categories are snapshots of the evaluated configuration, not live “safe corridor” volumes.

4.3 Model-assisted workflow

Slicer writes the selected CT to a private temporary workspace and starts the managed TotalSegmentator worker. The adapter maps available outputs to the CT grid, entry candidates are proposed, and exact finite paths are evaluated. The resulting state preserves `model_feasible`, `blocked`, `conditional`, `unavailable`, and `invalid` outcomes. `model_feasible` means that represented geometry did not block the configured finite instrument.

TotalSegmentator does not supply complete skull-base critical anatomy. Cranial nerves, cavernous-sinus contents, ophthalmic arteries, dura, and other structures can remain unavailable. The workflow displays these gaps as unavailable anatomy rather than treating them as free space.

5. Verification evidence and impact

Verification uses deterministic fixtures, independent geometric references, cross-platform automated tests, and programmatic desktop interactions. The release CI passed on Ubuntu, macOS, and Windows with Python 3.11 and 3.12. The scripted Slicer module was exercised with public CT data in 3D Slicer 5.12.4, while the positive worked example was verified programmatically. These tests establish software execution, not independent usability or clinical validation.

5.1 Positive worked example

The deterministic planning phantom contains an invented CT-like volume, a 413-sample ellipsoidal target, two finite portals, and three analytical protected spheres. It contains no patient data and is not an anatomical model. The same case was analyzed through the desktop workflow used for image-linked review. The EEA configuration produced 27 feasible sampled trajectories reaching 94 target samples; the transmaxillary configuration produced 77 trajectories reaching 267 samples. Set comparison assigned 59 samples to EEA only, 232 to transmaxillary only, 35 to both, 87 to sampled-unreached, and none to unavailable. The interface exposed 104 selectable trajectory witnesses and invalidated them when configuration state changed.

Deterministic positive synthetic worked example

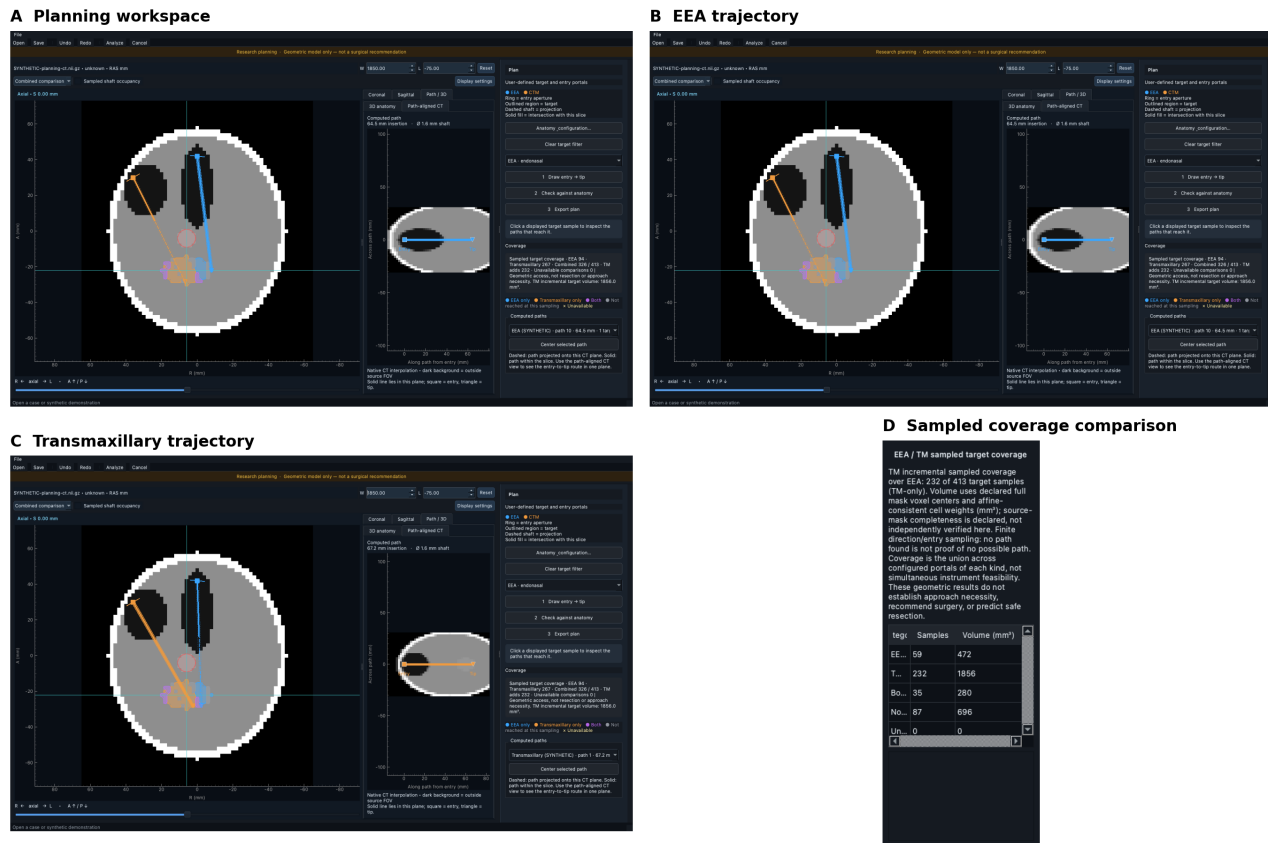


Figure 3. Positive synthetic worked example. (A) Linked-slice planning workspace. (B–C) Inspectable EEA and transmaxillary trajectory witnesses. (D) Sampled target-set comparison. All geometry is invented for software demonstration; the result is not anatomical evidence.

5.2 Numerical verification

For supported unobstructed analytical angular fixtures, an independent replay recorded 82 of 82 comparisons below a prespecified one-percent relative-error threshold; the maximum observed error was 0.0001204123%. This result applies to the geometry-aware unobstructed method, not protected-structure quadrature. In a separate legacy grid-quadrature stress fixture, a small off-axis target retained 11.368% error at the finest grid. That is a resolution limitation of the protected-geometry estimate: users should treat its solid-angle value as descriptive, inspect convergence, and omit the measurement when convergence is not demonstrated.

Voxel-cell refinement was first checked with 530 synthetic and 120 queries from three public cases, then repeated under a frozen protocol for 4,720 queries across 118 eligible NasalSeg cases (40 per case). A clearance-bound violation means the reported positive clearance exceeded the independent cell-box optimizer’s upper distance bound after subtracting instrument radius. No false-clear, clearance-bound, field-of-view-policy, or budget-fallback changes were observed. Refinement recovered 2,164 clear classifications that the coarse method conservatively blocked; 1,190 budget-exhausted queries retained the coarse blocked result. Because the public replay shares broad-phase candidate selection with the implementation, these observations test bounded numerical consistency rather than completeness.

NasalSeg v2 [20] supplied the public P001–P003 cases and the 118-case replay cohort. Twelve of 130 image/label pairs were excluded because image and label physical geometry did not match. Its air-space labels provide no operative-corridor reference standard; the dataset is used only for file, coordinate, and execution tests.

The software’s methodological contribution is the combination of inspectable finite-instrument witnesses, explicit unknown states, and set-based comparison of alternative versus simultaneous access. These capabilities support reproducible method studies and synthetic education scenarios.

6. Limitations

The model omits tissue deformation, dissection planes, endoscopic optics, handle access, hemostasis, reconstruction, staged debulking, and other operative factors. It neither infers complete neurovascular anatomy nor measures safe resectability. Results depend on target and portal definitions, registration, segmentation quality, target sampling, finite direction sampling, instrument parameters, fields of view, and conservative voxel approximations.

The public dataset does not provide operative-corridor reference annotations. TotalSegmentator predictions require review and do not represent all relevant anatomy. No cadaveric, tracked-instrument, operative-video, clinical-outcome, or independent surgeon-usability study is reported.

Native application redistribution requires review of the exact bundled binaries and dependency notices, including applicable Qt licensing obligations. Automated Python tests are cross-platform; independent human usability testing has not been performed.

The model-assisted workflow depends on a large local machine-learning stack and first-use model-weight retrieval. Installation time, disk use, and inference latency vary substantially by platform and accelerator. The managed runtime isolates these dependencies from Slicer’s embedded Python, but does not remove their upstream compatibility constraints.

7. Reproducibility and availability

The source code is licensed under Apache-2.0. Dependencies retain their own licenses. Version 0.2.1 is archived at [10.5281/zenodo.23244307](https://doi.org/10.5281/zenodo.23244307) and corresponds to Git tag `v0.2.1` at commit `a0c6933258f60e370747cfd4096b9445eff8a572`. The archive contains source, tests, documentation, packaging scripts, and deterministic fixtures; it excludes public CT archives, private data, model weights, and local sessions.

The source-release tooling uses an explicit allowlist, compares staged content with current files, records hashes, and runs wheel smoke checks. The Slicer package provisions an isolated per-user runtime without modifying Slicer’s embedded Python.

The retained NasalSeg manifest identifies version record 13893419, declared CC BY 4.0 terms, and archive SHA-256 `60c6facf843685802c39e4adff4a05c081c1c4b6175c9cb573745c55abb0fa6a`. The software does not redistribute the dataset.

Data availability

Source code version 0.2.1 is archived on Zenodo under DOI [10.5281/zenodo.23244307](https://doi.org/10.5281/zenodo.23244307) and developed publicly on GitHub. Synthetic fixtures and the worked-example generator are included in the repository. NasalSeg v2 is third-party data available from [Zenodo record 13893419](https://zenodo.org/record/13893419) under its stated terms and is not redistributed with the software. A separate persistent archive for the full development evidence is not part of the software release; the article reports the retained aggregate verification results.

8. Ethics and data governance

The software evidence uses synthetic fixtures and an existing public dataset described by its source as deidentified. It involves no recruitment, private records, chart review, outcomes, or identity linkage. Loyola University Chicago determined that this work did not require Institutional Review Board approval.

Images, predictions, temporary work products, Slicer scenes, and reports remain on the workstation unless the user explicitly exports them.

Hashes support integrity and traceability, not anonymization. User-entered case and approach identifiers may be identifying. Local paths, reviewer identities, registration filenames, screenshots, Slicer scenes, and visible annotations must be reviewed before sharing. Dataset licensing does not authorize reidentification or clinical claims.

9. Author contributions

Abhinav Bachu: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing—original draft, and Writing—review and editing.

Raghav Rajesh: Investigation, Visualization, Writing—original draft, and Writing—review and editing.

Anand V. Germanwala: Methodology, Validation, Supervision, and Writing—review and editing.

10. Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

11. Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

1. Fedorov A, Beichel R, Kalpathy-Cramer J, et al. 3D Slicer as an image computing platform for the Quantitative Imaging Network. *Magnetic Resonance Imaging*. 2012;30(9):1323–1341. doi:10.1016/j.mri.2012.05.001.
2. Ungi T, Lasso A, Fichtinger G. Open-source platforms for navigated image-guided interventions. *Medical Image Analysis*. 2016;33:181–186. doi:10.1016/j.media.2016.06.011.
3. Yeniaras E, Fuentes DT, Fahrenholtz SJ, et al. Design and initial evaluation of a treatment planning software system for MRI-guided laser ablation in the brain. *Int J CARS*. 2014;9:659–667. doi:10.1007/s11548-013-0948-x.
4. Muralidharan V, Swaminathan G, Devadhas D, Joseph BV. Patient-specific interactive software module for virtual preoperative planning and visualization of pedicle screw entry point and trajectories. *Neurology India*. 2018;66(6):1766–1770. doi:10.4103/0028-3886.246281.
5. Chen X, Xu L, Wang H, et al. Development of a surgical navigation system based on 3D Slicer for intraoperative implant placement surgery. *Medical Engineering & Physics*. 2017;41:81–89. doi:10.1016/j.medengphy.2017.01.005.

6. Narizzano M, Arnulfo G, Ricci S, et al. SEEG assistant: a 3DSlicer extension to support epilepsy surgery. *BMC Bioinformatics*. 2017;18:124. doi:10.1186/s12859-017-1545-8.
7. Brunenberg EJL, Vilanova A, Visser-Vandewalle V, et al. Automatic trajectory planning for deep brain stimulation: a feasibility study. *MICCAI*. 2007:584–592. doi:10.1007/978-3-540-75757-3_71.
8. Bériault S, Al Subaie F, Collins DL, Sadikot AF, Pike GB. A multi-modal approach to computer-assisted deep brain stimulation trajectory planning. *Int J CARS*. 2012;7(5):687–704. doi:10.1007/s11548-012-0768-4.
9. Shamir RR, Joskowicz L, Tamir I, et al. Reduced risk trajectory planning in image-guided keyhole neurosurgery. *Medical Physics*. 2012;39(5):2885–2895. doi:10.1118/1.4704643.
10. Trope M, Shamir RR, Joskowicz L, et al. The role of automatic computer-aided surgical trajectory planning in improving the expected safety of stereotactic neurosurgery. *Int J CARS*. 2015;10(7):1127–1140. doi:10.1007/s11548-014-1126-5.
11. Sparks R, Vakharia V, Rodionov R, et al. Anatomy-driven multiple trajectory planning of intracranial electrodes for epilepsy surgery. *Int J CARS*. 2017;12(8):1245–1255. doi:10.1007/s11548-017-1628-z.
12. Wankhede A, Madiraju L, Siampali E, et al. Validation of a novel path planner for stereotactic neurosurgical interventions. *Int J Med Robotics Comput Assist Surg*. 2022;18(6):e2458. doi:10.1002/rcs.2458.
13. Roth J, Singh A, Nyquist G, et al. Three-dimensional and 2-dimensional endoscopic exposure of midline cranial base targets. *Neurosurgery*. 2009;65(6):1116–1128. doi:10.1227/01.NEU.0000360340.85186.7A.
14. Wilson DA, Williamson RW, Preul MC, Little AS. Comparative analysis of surgical freedom and angle of attack of two minimal-access endoscopic transmaxillary approaches. *World Neurosurgery*. 2014;82:e487–e493. doi:10.1016/j.wneu.2013.02.003.
15. Elhadi AM, Almefty KK, Mendes GAC, et al. Comparison of surgical freedom and area of exposure in three endoscopic transmaxillary approaches. *J Neurol Surg B Skull Base*. 2014;75(5):346–353. doi:10.1055/s-0034-1372467.
16. Lin B-J, Ju D-T, Hsu T-H, et al. Quantitative comparison of endoscopically assisted endonasal, sublabial and transorbital transmaxillary approaches. *Clinical Otolaryngology*. 2021;46(1):123–130. doi:10.1111/coa.13559.
17. Agosti E, Saraceno G, Rampinelli V, et al. Quantitative anatomic comparison of endoscopic transnasal and microsurgical transcranial approaches. *Operative Neurosurgery*. 2022;23(4):e256–e266. doi:10.1227/ons.0000000000000312.
18. Wasserthal J, Breit H-C, Meyer MT, et al. TotalSegmentator: robust segmentation of 104 anatomic structures in CT images. *Radiology: Artificial Intelligence*. 2023;5(5):e230024. doi:10.1148/ryai.230024.
19. Isensee F, Jaeger PF, Kohl SAA, Petersen J, Maier-Hein KH. nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation. *Nature Methods*. 2021;18(2):203–211. doi:10.1038/s41592-020-01008-z.
20. NasalSeg dataset, version record 13893419. Zenodo. <https://zenodo.org/records/13893419> (accessed 8 October 2026).